

Modelos open source en tu IDE.

Qué tipos de modelo podemos usar y cómo nos ayudan a optimizar nuestros costos mientras mantenemos la gobernanza de nuestros datos.

Glosario

Antes de comparar opciones, alineemos vocabulario. No es lo mismo el modelo, la app que usás, ni cómo lo integrás en código.

Modelo

Red entrenada con pesos y arquitectura (GPT, Llama, Claude...)

API

Interfaz para invocar el modelo desde código o sistemas

Herramienta

Interfaz de usuario sobre el modelo (chat, asistente)

Plataforma

Producto listo para usar (ChatGPT, Copilot, Cursor)

¿Qué tan **abierto** es el modelo?

Propietario cerrado

Pesos privados. Solo acceso por API o suscripción

GPT-5.x, Claude, Gemini

Open weights

Pesos descargables, con condiciones de licencia

Llama 4, Mistral, Qwen, DeepSeek

Modelo custom

Fine-tuned o entrenado con datos propios. Pesos no públicos

Adaptación de Llama, modelo interno

Open source real

Pesos + receta de entrenamiento + datos abiertos

OLMo, modelos de investigación

¿Dónde corren tus datos?

El mismo Llama puede estar en tu laptop o en la nube: la apertura del modelo y el despliegue son decisiones independientes.

+ CONTROL / + PRIVACIDAD

+ CONVENIENCIA / + EXPOSICIÓN

Tu máquina	Servidor on-prem	Nube privada	Hosteado por tercero	API del proveedor	App pública (consumer)
Laptop o workstation con Ollama / LM Studio. Nada sale del equipo.	GPU propia sirviendo al equipo con vLLM, dentro de tu red.	Tu cuenta en la nube, red privada y políticas tuyas.	Open weights en infra compartida: Groq, Together, Replicate.	Endpoint con contrato y DPA (GPT-5.x, Claude).	ChatGPT free, apps sueltas. Sin garantías de datos.
CONTROL	CONTROL	CONTROL	CONTROL	CONTROL	CONTROL
PRIVACIDAD	PRIVACIDAD	PRIVACIDAD	PRIVACIDAD	PRIVACIDAD	PRIVACIDAD
ESFUERZO	ESFUERZO	ESFUERZO	ESFUERZO	ESFUERZO	ESFUERZO
Alto	Alto	Medio	Medio	Bajo	Mínimo
Alta	Alta	Alta	Media	Variable	Baja
Bajo	Alto	Medio	Bajo	Bajo	Nulo

No elegís solo el **modelo**. Elegís el **sistema**.

Este sistema define tu privacidad, qué licencia comercial te limita, si podés escalar y a qué costo, qué herramientas podés usar, la inteligencia de los modelos, etc.

A raíz de esto surgen flujos híbridos que son capaces de utilizar lo mejor de ambos mundos, combinan parte de tareas realizadas en local y otra parte en IA de frontera.

USOS COMUNES DEL APPROACH HÍBRIDO

Anonimizar información

Detectás PII en local, reemplazás con tokens y guardás el mapa. Solo mandás texto limpio a la frontera; al volver, rehidratás con el mapa que nunca salió.

Pre-procesar para ahorrar tokens

OCR, extracción o clasificación en local. Llegás a Claude o GPT con data ya procesada y una query más chica.

Comparativa: calidad y costo

	PROPIETARIO	LOCAL / OPEN	HÍBRIDO
Calidad	Tope en tareas complejas	La brecha se achicó en 2026 (Llama, Qwen, DeepSeek)	Razonás en el mejor modelo, el dato lo preparás local
Privacidad	Según contrato (DPA)	Datos en tu perímetro	Anonimizás local, mandás solo lo limpio
Costo al arrancar	Bajo: API o suscripción	Alto: GPU y setup	Medio: API más una capa local
Costo a escala	Sube con el volumen	Se amortiza a volumen alto	Lo barato local, lo crítico por API
Tiempo de implementación	Rápido	Lento	Medio

Comparativa: operación

PROPIETARIO

LOCAL / OPEN

Mantenimiento	Del proveedor	Propio
Escalabilidad	Gestionada por el vendor	La construís vos
Gobernanza	Políticas del proveedor	Políticas internas
Soporte	Contrato comercial (SLA)	Comunidad o consultoría
Trazabilidad	Logs del proveedor	Logs propios

Sugerencias de modelos por tarea

LOCAL / OPEN

API / SUSCRIPCIÓN

Razonamiento complejo	DeepSeek V4, Qwen 3.6 grande	Claude Opus, GPT-5.x, Gemini Pro
Coding y agentes	Qwen 3.6 Coder, Devstral	Claude Opus, GPT-5.x
Clasificar / extraer	Qwen chico, Gemma 4, Phi	Gemini Flash, GPT-5 mini
Resúmenes simples	Llama 4, Mistral	Gemini Flash, Claude Haiku
Multimodal / visión	Qwen-VL, Gemma 4	Gemini, GPT-5.x
Generar imágenes	FLUX, Stable Diffusion	GPT Image, Imagen, Midjourney
Embeddings (RAG)	nomic-embed, bge	OpenAI, Voyage, Cohere

¿Cuándo usar **open** vs **frontera**?

OPEN / LOCAL

Datos sensibles: PII, salud, legal, financiero

Los datos no deberían salir de tu red

Menos dependencia de un proveedor externo

Tareas acotadas: clasificar, extraer, etiquetar

Resúmenes simples con un modelo chico

Prototipos internos con control de acceso

FRONTERA / PROPIETARIO

Razonamiento complejo donde importa la calidad

Producto listo, sin montar infraestructura

Integraciones con M365, Google, Slack, etc.

Necesitás salir rápido a producción

Datos no sensibles, o con DPA

Sin equipo que opere modelos (MLOps)

Conectar LLMs locales a tu IDE

VS CODE

1. Instalá la extensión oficial **Ollama** desde el marketplace.
2. La extensión usa tus modelos locales en el endpoint por defecto `http://localhost:11434`.
3. En el chat, elegí el modelo que quieras en la sección **Ollama**.

SI APARECE ERROR BYOK

Settings → buscá `BYOK utility model default` → cambiá a **Main Agent Model**.

Cómo leer el **nombre** de un modelo

qwen2.5:7b-instruct-q4

FAMILIA

Quién lo entrenó y qué arquitectura usa.

ESCALA

Cantidad de parámetros.
Más = más capacidad y más memoria.

AJUSTE

Afinado para seguir instrucciones y chatear.

CUANTIZACIÓN

Precisión de los pesos.
Menos bits = menos memoria.

030

3B, 7B, 14B, 70B son modelos distintos, de menor a mayor capacidad. La **cuantización** no cambia esa cifra: es el mismo modelo guardado con menos precisión.

Cuantización: **el mismo modelo**, menos memoria

Un mismo 7B ocupa muy distinto según con cuánta precisión guardes sus pesos.

FP16 • SIN CUANTIZAR

≈ 14 GB

Precisión completa. La referencia de calidad.

Q8

≈ 7–8 GB

Casi sin pérdida perceptible.

Q4

≈ 4–5 GB

El default práctico para correr local.

No es gratis: a menos bits, algo de precisión se pierde. Para extraer campos o clasificar casi no se nota; para razonamiento fino, sí.

RAM y VRAM: qué tiene que entrar

El modelo corre rápido si entra en memoria. Dónde entra define tu velocidad.

GPU + VRAM

Si el modelo entra en la VRAM, la GPU infiere rápido.

Es el camino ideal, pero la VRAM suele ser el recurso más escaso.

CPU + RAM

Sin GPU, corre en RAM sobre CPU: funciona, pero más lento.

Ollama puede repartir capas entre GPU y CPU si no entra todo.

REGLA PRÁCTICA (CON MARGEN PARA SO Y CONTEXTO)

8 GB

1B – 3B en Q4

16 GB

7B – 8B en Q4

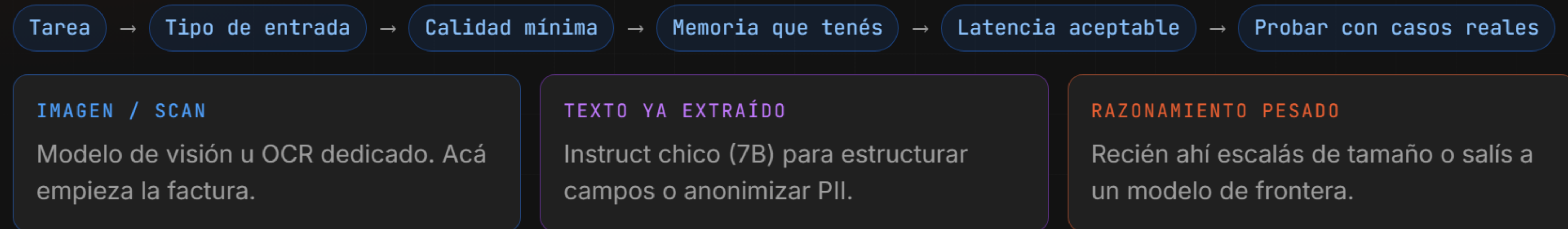
32 GB

14B en Q4

No entran solo los pesos: también el runtime y el **contexto** (KV cache). Prompts largos comen memoria. Verificá con **ollama ps**.

El modelo **más chico** que resuelve bien la tarea

No arranques por el ranking. Arrancá por lo que tenés que hacer.



No elegimos por ranking: elegimos por **tarea**, **evidencia** y **hardware**.

Gracias.